HDOC DAY – 72

Algorithm

Step 1: Start.

Step 2: Read the initial number of students and number of working days.

Step 3: Check that both values are greater than zero.

Step 4: Call createMatrix() and dynamically allocate memory using:

malloc(students * days * sizeof(int))

Step 5: Call recordAttendance() to enter attendance for every student and day.

Step 6: Validate every attendance value. Accept only:

- 1 → Present
- 0 → Absent

Step 7: Display the initial attendance matrix using displayAttendance().

Step 8: Read the number of additional students.

Step 9: Call addStudents() to increase the matrix size using realloc().

Step 10: Record attendance for the newly added students.

Step 11: Display the updated attendance matrix.

Step 12: Call calculatePercentage() for each student:

\[\text{Attendance Percentage} = \frac{\text{Number of Present Days}}{\text{Total Working Days}} \times 100 \]

Step 13: Call releaseMemory() to release the dynamically allocated memory using free().

Step 14: Stop.

C program:

```
#include <stdio.h>

#include <stdlib.h>
```

```
/* Function prototypes */
int *createMatrix(int students, int days);
void recordAttendance(int *attendance, int start, int students, int days);
int *addStudents(int *attendance, int *students, int newStudents, int days);
void displayAttendance(int *attendance, int students, int days);
void calculatePercentage(int *attendance, int students, int days);
void releaseMemory(int *attendance);


/* Create initial attendance matrix */
int *createMatrix(int students, int days)
{
   int *attendance;

   attendance = (int *)malloc(students * days * sizeof(int));

   if (attendance == NULL)
   {
      printf("Memory allocation failed.\n");
      return NULL;
   }

   return attendance;
}
```

```
/* Record attendance */
void recordAttendance(int *attendance, int start, int students, int days)
{
   int i, j, value;

   for (i = start; i < students; i++)
   {
      printf("\nEnter attendance for Student %d:\n", i + 1);

      for (j = 0; j < days; j++)
      {
         do
         {
            printf("Day %d (0-Absent, 1-Present): ", j + 1);

            if (scanf("%d", &value) != 1)
            {
               printf("Invalid input. Enter only 0 or 1.\n");

               while (getchar() != '\n');

               value = -1;
            }
```

```
            else if (value != 0 && value != 1)
            {
               printf("Invalid attendance. Enter only 0 or 1.\n");
            }

         } while (value != 0 && value != 1);

         *(attendance + i * days + j) = value;
      }
   }
}


/* Add new students using realloc() */
int *addStudents(int *attendance, int *students,
              int newStudents, int days)
{
   int oldStudents;
   int totalStudents;
   int *temp;

   oldStudents = *students;
   totalStudents = oldStudents + newStudents;

   temp = (int *)realloc(attendance,
```

```
                totalStudents * days * sizeof(int));

    if (temp == NULL)
    {
        printf("Memory reallocation failed.\n");
        return attendance;
    }

    attendance = temp;
    *students = totalStudents;

    /* Record attendance only for newly added students */
    recordAttendance(attendance, oldStudents, totalStudents, days);

    return attendance;
}


/* Display attendance matrix */
void displayAttendance(int *attendance, int students, int days)
{
    int i, j;

    printf("\nAttendance Records:\n\n");
```

```
    printf("Student\t");

    for (j = 0; j < days; j++)
    {
        printf("Day %d\t", j + 1);
    }

    printf("\n");

    for (i = 0; i < students; i++)
    {
        printf("Student %d\t", i + 1);

        for (j = 0; j < days; j++)
        {
            printf("%d\t", *(attendance + i * days + j));
        }

        printf("\n");
    }
}


/* Calculate attendance percentage */
void calculatePercentage(int *attendance, int students, int days)
```

```
{
    int i, j, present;
    float percentage;

    printf("\nAttendance Percentage:\n");

    for (i = 0; i < students; i++)
    {
        present = 0;

        for (j = 0; j < days; j++)
        {
            if (*(attendance + i * days + j) == 1)
            {
                present++;
            }
        }

        percentage = ((float)present / days) * 100;

        printf("Student-%d: %.1f%%\n", i + 1, percentage);
    }
}
```

```
/* Release dynamically allocated memory */
void releaseMemory(int *attendance)
{
   free(attendance);
}


int main()
{
   int *attendance;
   int students, days;
   int newStudents;

   printf("Enter initial number of students: ");
   scanf("%d", &students);

   printf("Enter number of working days: ");
   scanf("%d", &days);

   /* Validate dimensions */
   if (students <= 0 || days <= 0)
   {
      printf("Invalid number of students or working days.\n");
      return 1;
   }
```

```
/* Create initial matrix */
attendance = createMatrix(students, days);

if (attendance == NULL)
{
   return 1;
}

/* Record initial attendance */
recordAttendance(attendance, 0, students, days);

/* Display initial matrix */
displayAttendance(attendance, students, days);

printf("\nEnter number of additional students: ");
scanf("%d", &newStudents);

if (newStudents < 0)
{
   printf("Invalid number of students.\n");
   releaseMemory(attendance);
   return 1;
}
```

```
    /* Add new students */
    if (newStudents > 0)
    {
        attendance = addStudents(attendance, &students,
                                 newStudents, days);
    }

    /* Display final matrix */
    displayAttendance(attendance, students, days);

    /* Calculate percentages */
    calculatePercentage(attendance, students, days);

    /* Free allocated memory */
    releaseMemory(attendance);

    return 0;
}
```

Test Case Table

Test	Initial Students	Days	Additional Students	Attendance	Expected Percentage
1	3	3	3	100, 101, 111, 100, 111, 101	33.3%, 66.7%, 100%, 33.3%, 100%, 66.7%

2	2	4	0	1111, 1010	100%, 50%
3	1	5	1	00000, 11111	0%, 100%
4	0	3	—	—	Invalid number of students
5	3	0	—	—	Invalid number of working days
6	2	3	0	Attendance input 2	Reject and ask again